

INFORME FULL DETALLADO

PRUEBAS DE SEGURIDAD WEB Y MITIGACIÓN

DE VULNERABILIDADES

SIHUL

Sistema Integral de Horarios, Usuarios, Préstamos, Académico y Financiero

Campo	Detalle
Fecha de elaboración	31 de mayo de 2026
Período cubierto	27 al 31 de mayo de 2026
Entorno principal de prueba	Docker local: Backend/API http://localhost:8000; Nginx local http://localhost:8081
Tecnologías	Django 5.2.7 + DRF, React + Vite, PostgreSQL, FastAPI Chatbot, Nginx, Docker
Estado	Mitigaciones críticas aplicadas; pendiente validación final en staging/HTTPS y ejecución recurrente de ZAP/CI
Versión del informe	2.0 - versión ampliada y detallada

Resumen de alcance

Este documento consolida las pruebas de autenticación, sesiones, APIs expuestas, IDOR, CSRF, XSS, carga de archivos, dependencias, secretos, configuración, SQL Injection puntual, ZAP Baseline y controles operativos implementados para SIHUL.

Tabla de contenido

1. 1. Resumen ejecutivo ampliado
2. 2. Alcance técnico y entorno de pruebas
3. 3. Metodología de pruebas aplicada
4. 4. Mapa consolidado de vulnerabilidades y mitigaciones
5. 5. Pruebas de autenticación, sesiones y APIs expuestas
6. 6. Blindaje IDOR - SIHUL Académico y Financiero
7. 7. Protección CSRF
8. 8. Protección XSS
9. 9. Carga de archivos y manejo seguro de adjuntos
10. 10. SQL Injection y validación de consultas
11. 11. Dependencias, CI/CD y vulnerabilidades de paquetes
12. 12. Secretos, configuración y endurecimiento Docker/Nginx
13. 13. OWASP ZAP, headers y controles perimetrales
14. 14. Matriz detallada de hallazgos
15. 15. Matriz de pruebas realizadas
16. 16. Riesgos residuales y recomendaciones finales
17. 17. Anexos técnicos

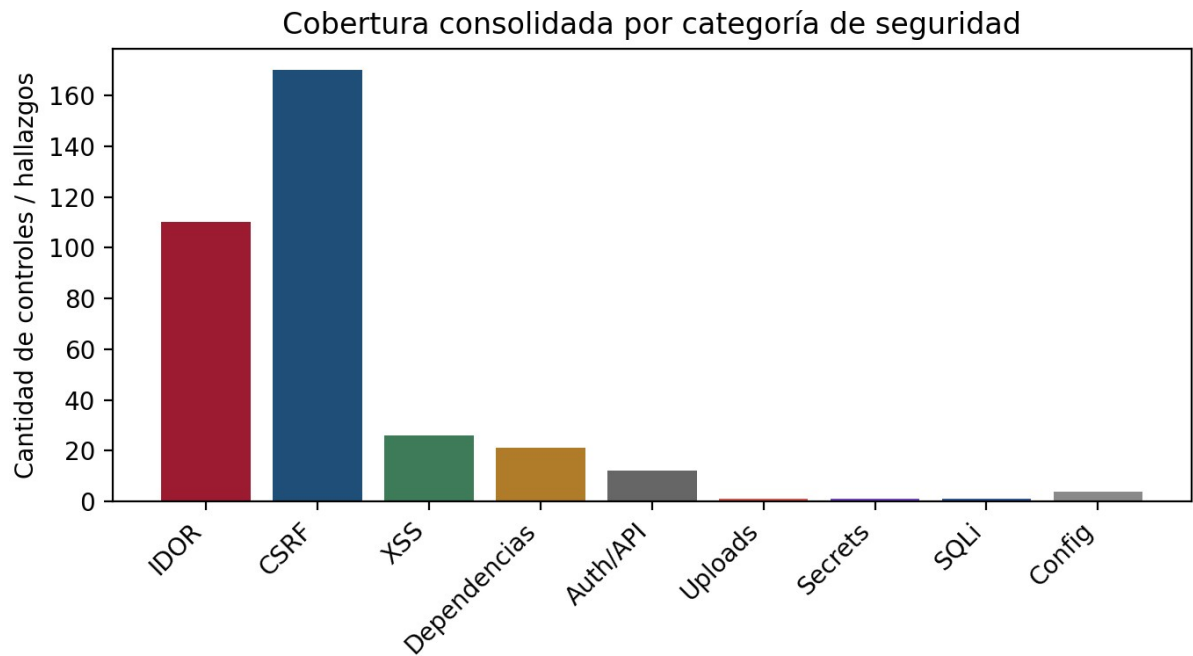
1. Resumen ejecutivo ampliado

Durante el ciclo de revisión y hardening de SIHUL se identificaron debilidades críticas y altas que afectaban la autenticación, el control de acceso, la exposición de APIs, la protección CSRF, el manejo de entradas, la carga de archivos, las dependencias, los secretos y la configuración de despliegue. El trabajo no consistió únicamente en reportar vulnerabilidades, sino en comprobarlas, corregirlas, volver a probarlas y documentar el patrón de mitigación aplicado.

La revisión cubrió SIHUL Académico, SIHUL Financiero, usuarios, préstamos, componentes, notificaciones, chatbot, frontend, Nginx, Docker y flujos públicos protegidos por reCAPTCHA/token público.

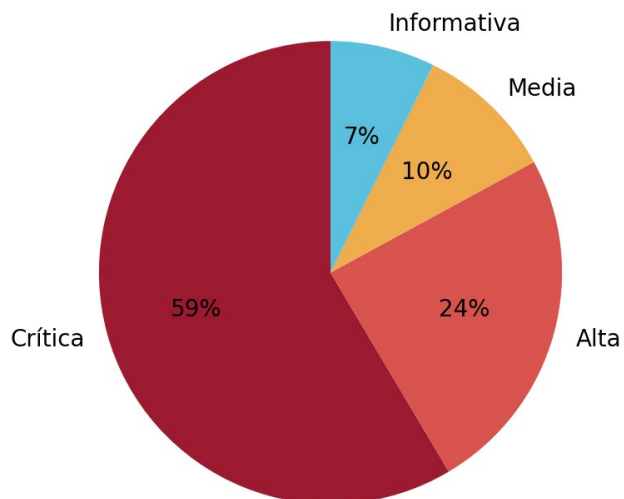
Métrica	Resultado consolidado
Vulnerabilidades críticas documentadas y mitigadas	32
CVEs / vulnerabilidades de dependencias abordadas	21
Endpoints protegidos contra IDOR	110+
Instancias CSRF cubiertas	170+
Endpoints protegidos contra XSS	26
Campos sanitizados	100+
Esquemas de validación XSS	17
Módulos principales incluidos	Core, Académico, Financiero, Usuarios, Préstamos, Chatbot
Estado operativo	Sistema funcional con controles de seguridad aplicados

Gráfica 1. Cobertura consolidada por categoría.



Gráfica 2. Distribución estimada de severidad.

Distribución estimada de severidad documentada



Conclusión ejecutiva

SIHUL pasó de un estado con múltiples superficies expuestas a una arquitectura con controles centralizados de autenticación, autorización, ownership, CSRF, sanitización, validación de archivos, auditoría de dependencias, manejo de secretos y verificación continua. El informe recomienda no declarar cierre absoluto hasta validar en staging/HTTPS con ZAP, CI verde, secretos rotados y pruebas E2E ejecutadas.

2. Alcance técnico y entorno de pruebas

El alcance incluyó pruebas locales en Docker y revisión del comportamiento de los endpoints expuestos por backend directo y por Nginx local.

Elemento	Detalle
Frontend	React + Vite; Vite dev server http://localhost:5173 ; build servido por Nginx local
Backend	Django 5.2.7 + Django REST Framework en http://localhost:8000
Proxy local	Nginx local http://localhost:8081 para simular despliegue de frontend y headers
Base de datos	PostgreSQL en Docker
Chatbot	FastAPI/Uvicorn, con dependencias actualizadas
Ambiente interno	sihul.unilibre.edu.co accesible desde red interna universitaria
Usuarios de prueba	Académicos: admin, admin_planeacion, docente, planeacion, supervisor, karol.docente; Financieros: proveedor, funcionario, contabilidad, tesorería, auditoría, dirección financiera, rectoría, admin financiero

2.1 Criterio para usar 8000 y 8081

Para pruebas directas de vulnerabilidad backend/API, especialmente IDOR, se trabajó principalmente contra <http://localhost:8000>, porque permite probar rutas DRF y legacy reales sin depender del enrutamiento del frontend. Para pruebas de usuario final, Nginx, headers y ZAP se priorizó <http://localhost:8081>.

Tipo de prueba	URL preferida	Justificación
IDOR / ownership / API	http://localhost:8000	Endpoint real del backend, incluyendo rutas legacy y DRF
ZAP / headers / frontend build	http://localhost:8081	Simula mejor producción mediante Nginx
Flujos visuales de usuario	http://localhost:8081	Permite revisar UX, bloqueo de login, cookies y navegación
Desarrollo rápido	http://localhost:5173	Solo para desarrollo Vite; no representa producción

3. Metodología de pruebas aplicada

La metodología se ejecutó en ciclos: identificación, reproducción, evidencia, mitigación, reinicio/rebuild cuando aplicaba, reprueba y documentación.

- Levantamiento del ambiente local Docker y verificación de contenedores.
- Exploración manual con navegador, DevTools, Network y endpoints directos.
- Pruebas con usuarios de distintos roles para validar autorización y exposición de datos.
- Uso de OWASP ZAP Desktop/Baseline como verificación pasiva y detección de headers/cookies.
- Revisión de código en endpoints DRF, vistas legacy, serializers, middleware, settings y docker-compose.
- Mitigación de hallazgos mediante patrones reutilizables: admin-only, same-user-or-admin, seccional filtering, role-based access, token público y sanitización centralizada.
- Reprueba manual con los mismos endpoints y usuarios para confirmar 403/404/200 esperados.
- Consolidación documental con evidencias, riesgos residuales y próximos pasos.

3.1 Criterios de severidad

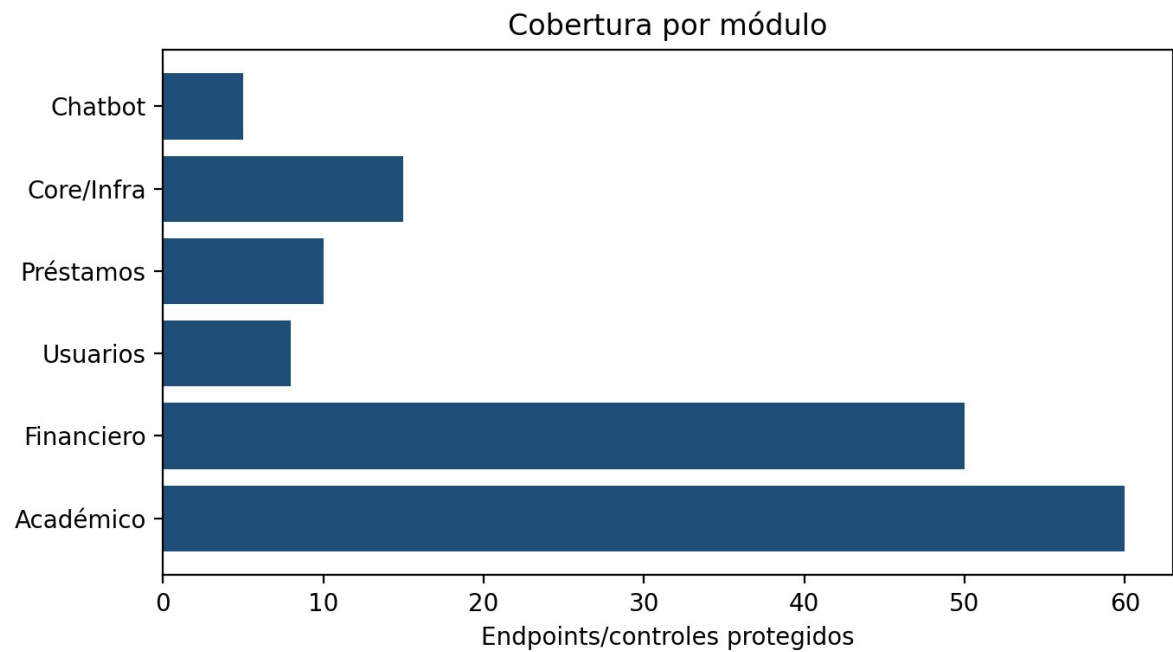
Severidad	Criterio aplicado	Ejemplos SIHUL
Crítica	Permite acceso no autorizado, toma de control, exposición masiva o bypass de protección esencial	IDOR en préstamos, exposición de usuarios/ hashes, CSRF en mutaciones, secretos en repo, dependencias críticas
Alta	Exposición significativa o modificación no autorizada bajo condiciones realistas	Catálogos públicos, XSS almacenado, uploads inseguros, headers ausentes
Media	Riesgo explotable con condiciones adicionales o mitigado parcialmente	Rate limit visual no persistente, metadata excesiva, mensajes de error
Baja/Informativa	Mejora de hardening o documentación sin explotación directa confirmada	Secure pendiente en HTTP local, ZAP baseline pendiente de ejecución final

4. Mapa consolidado de vulnerabilidades y mitigaciones

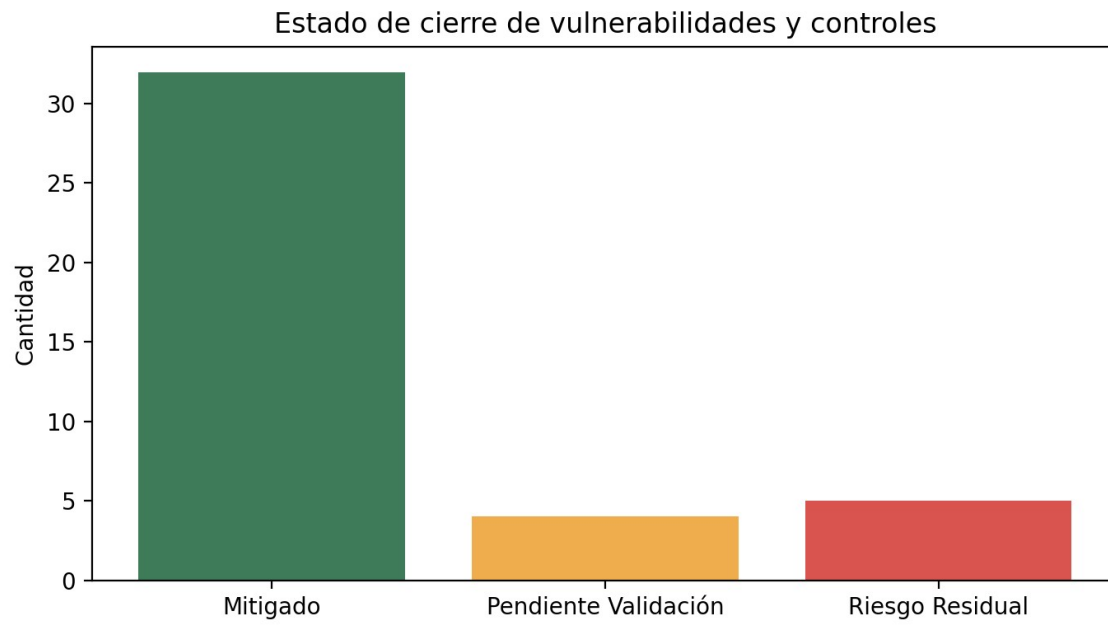
A continuación se presenta una visión ejecutiva de los principales frentes cubiertos.

Categoría	Hallazgo principal	Mitigación implementada	Estado
Autenticación	Login real /api/usuarios/login/ sin logout efectivo; errores 405/403 por configuración	Corrección de VITE_API_URL, CSRF allowlist para login, logout 2 minutos, botón deshabilitado con contador y localStorage	Mitigado
Sesiones	Necesidad de validar cookies y cierre de sesión	sessionid HttpOnly, SameSite=Lax; logout invalida sesión; Secure pendiente en HTTPS	Mitigado local / pendiente HTTPS
APIs expuestas	Endpoints públicos devolvían datos sensibles sin autenticación	Autenticación obligatoria, cierre de GET público en /api/prestamos/publicos/, filtros por identificación/correo, 403 en incógnito	Mitigado
IDOR	Acceso a recursos por cambio de ID en usuarios, préstamos, horarios, espacios y financiero	Admin-only, same-user-or-admin, seccional filtering, role-based access, ownership validation	Mitigado
CSRF	170 instancias @csrf_exempt y mutaciones sin protección central	JSONCsrfMiddleware, X-CSRFToken, endpoint /api/csrf-token/, allowlist estricta	Mitigado
XSS	Inputs y serializers sin sanitización robusta	xss_protection.py, sanitize_dict, 17 schemas, whitelists, 26 endpoints protegidos	Mitigado
Uploads	Carga de archivos financieros con riesgo de malware/path traversal	Extensión, MIME, magic bytes, tamaño 10MB, UUID, sanitización de nombre	Mitigado
Dependencias	21 vulnerabilidades HIGH/CRITICAL y checks fallando	Actualización de paquetes, pins, pip-audit/safety/npm audit en CI, FastAPI/Starlette actualizado	Mitigado / xlsx a vigilar si aplica
Secretos	Claves y contraseñas hardcodeadas en compose	Variables de entorno, .env ignorado, .env.example, DJANGO_SECRET_KEY requerido en producción, rotación recomendada	Mitigado con pendiente rotación
SQLi	Comando Oracle aceptaba query cruda desde CLI	Validación SELECT-only, bloqueo de múltiples sentencias y keywords peligrosas	Mitigado
ZAP/Headers	Necesidad de verificación pasiva de headers/cookies/CORS	docker-compose.zap.yml y workflow ZAP Baseline; pendiente revisar reporte final	Implementado / pendiente ejecutar final

Gráfica 3. Cobertura de controles por módulo.



Gráfica 4. Estado de cierre y riesgos residuales.



5. Pruebas de autenticación, sesiones y APIs expuestas

5.1 Login y errores de configuración

ID	Hallazgo	Evidencia/Síntoma	Mitigación	Validación
AUTH-01	Error 405 en login por comillas literales en variables VITE	POST a /%22http://localhost:8000%22/api/usuarios/login/ generaba 405	Remoción de comillas en docker-compose: VITE_API_URL=\${VITE_API_URL:-http://localhost:8000}	Login responde 200 con credenciales válidas
AUTH-02	Login público bloqueado por CSRF	POST /api/usuarios/login/ retornaba 403 CSRF token missing	CSRF_EXEMPT_PATHS para login/logout/csrf-token y respeto a decoradores csrf_exempt	Login vuelve a funcionar sin romper protección en mutaciones
AUTH-03	Endpoint DRF de login sin logout	Intentos ilimitados con 401 y sin 429	Lockout en /api/usuarios/login/ y cache local; límite de intentos y bloqueo 2 minutos	Luego de intentos fallidos responde 429/bloqueo
AUTH-04	Bloqueo visual evadible al recargar	Al refrescar página se podía volver a intentar	localStorage con logout_until; botón deshabilitado, opacidad y contador	Recarga mantiene bloqueo hasta 120s

5.2 Pruebas de sesión y cookies

Prueba	Resultado esperado	Resultado obtenido/documentado	Estado
Cookie sessionid	HttpOnly=true; SameSite=Lax; Secure en HTTPS	HttpOnly y SameSite=Lax en local; Secure no aplica en HTTP local	Aprobado local
Logout	Después de logout, endpoint de usuario debe devolver no autenticado	/api/auth/user/ devuelve {error: No autenticado}	Aprobado
Acceso admin con usuario normal	Usuario no admin no debe entrar al panel	Redirección a login del admin	Aprobado
API auth state	Debe leerse con endpoint correcto de sesión usada por SIHUL	Se aclaró diferencia entre /api/auth/user/ y /usuarios/session-auth-state/	Aprobado

5.3 APIs expuestas y rutas públicas

Endpoint	Problema detectado	Decisión de seguridad	Resultado esperado
GET /api/prestamos/publicos/	Permitía entrar a API sin autenticación y consultar información pública sensible	GET requiere autenticación; solo POST público con reCAPTCHA	Incógnito -> 403
Pantalla Mis Solicitudes	Cuando usuario autenticado consultaba con identificación/correo, backend devolvía todo si no filtraba correctamente	Filtrar por identificación + correo; solo admin lista todo	Usuario normal ve solo sus solicitudes
/api/prestamos/tipos-actividad/	GET público por diseño	Se mantiene público si solo entrega datos mínimos no sensibles	200 con datos mínimos
/api/prestamos/public/espacios-disponibles/	Público por diseño para flujo de préstamo público	Permitir únicamente disponibilidad mínima necesaria	200 con información no sensible
/prestamos/list/ y /prestamos/public/mis-solicitudes/	Legacy con posible exposición	Requiere auth/token según flujo	Incógnito -> 403
/periodos/*	Posible exposición de periodos	Requiere autenticación para rutas internas	Incógnito -> 403
/horario/por-periodo/	Handler detectado sin auth en revisión	Agregar auth o cerrar si devuelve datos de horarios	Sin login -> 403

6. Blindaje IDOR - SIHUL Académico y Financiero

IDOR fue uno de los frentes más importantes. Se probó cambio manual de IDs en URLs, comparación entre usuarios normales y administradores, y consultas cruzadas en préstamos, horarios, usuarios y módulos financieros.

6.1 Patrones de mitigación implementados

Patrón	Uso	Regla implementada
Admin-only	Creación, actualización y eliminación de catálogos/configuración	Solo admin global/sistema/financiero puede mutar
Same-user-or-admin	Datos personales, horarios, endpoints con usuario_id	Usuario solo accede a su ID; admin accede a todos
Seccional filtering	Académico: sedes, facultades, programas, grupos, espacios	Usuario normal solo ve su seccional/sede; admin ve todo
Role-based filtering	Financiero: proveedor, funcionario, contabilidad, tesorería, auditoría, dirección	Queryset depende del rol y flujo de negocio
Ownership validation	Facturas, documentos, comentarios, préstamos	El recurso solicitado debe pertenecer al usuario o a su ámbito autorizado
Token público	Préstamos públicos	Consulta/modificación requiere X-Public-Token correcto

6.2 Pruebas IDOR académicas ejecutadas

Caso	Usuario	URL / recurso	Resultado esperado	Estado
Usuarios - propio	Karol docente (id 652)	/api/usuarios/652/	200	Correcto
Usuarios - ajeno/admin	Karol docente	/api/usuarios/2/ (admin)	403/404	Correcto
Préstamo propio	Karol docente	/api/prestamos/espacios/1/ y /2/	200	Correcto
Préstamo ajeno	Karol docente	/api/prestamos/espacios/3/ y /4/	Inicialmente 200, luego 403/404	Mitigado
Horario propio	Karol docente	/horario/mi-horario-docente? usuario_id=652	200	Mitigado/validar final
Horario ajeno	Karol docente	/horario/mi-horario-docente? usuario_id=5	403/404	Mitigado/validar final
Espacios legacy sin login	Incógnito	/espacios/list/ y /espacios/1/	403	Mitigado
Espacios autenticado	Usuario normal	/espacios/*	Solo lectura permitida; escritura admin-only	Mitigado

6.3 Endpoints académicos protegidos

Módulo	Endpoints / acciones	Vulnerabilidad	Mitigación
Usuarios	/api/usuarios/, /api/usuarios/{id}/, /usuarios/list/, /usuarios/get/{id}	Exposición de usuarios, acceso a IDs ajenos, hashes	Perfil mínimo, same-user-or-admin, admin-only en escritura
Grupos	GET/POST/PUT/DELETE /grupos/{id}	Lectura/escritura sin rol	Auth + seccional filtering; admin-only escritura
Asignaturas	GET/POST/PUT/DELETE /asignaturas/{id}	Acceso por ID sin validación	Auth + seccional filtering; admin-only escritura
Programas	GET/PUT /programas/{id}	Acceso a programas de otras seccionales	Filtrado por seccional
Facultades	GET/PUT /facultades/{id}	Acceso o modificación sin autorización	Filtrado por seccional y admin-only
Sedes	GET /sedes/{id}	Acceso no filtrado	Auth + seccional filtering
Horarios	/horario/mi-horario-docente?usuario_id=, /horario/mi-horario-estudiante?usuario_id=, exportaciones por usuario	Cambio de usuario_id para ver horario ajeno	Same-user-or-admin
Espacios	/espacios/*, /espacios/permitido/*, /espacios/horarios/*	Endpoints legacy públicos y enumerables	Auth obligatoria; supervisor/dueño/admin según caso
Préstamos	/api/prestamos/espacios/{id}, /api/prestamos/recursos/	Consulta de préstamos ajenos	Queryset por usuario; admin ve todos
Notificaciones	legacy views y DRF serializer	IDOR + XSS en mensaje/tipo	Auth obligatoria, ownership, sanitización

6.4 Endpoints financieros protegidos contra IDOR

Módulo financiero	Endpoints / acciones	Riesgo	Mitigación
Proveedores	GET/POST/PUT /proveedores, /proveedores/{id}	Exposición o modificación por rol no autorizado	Role-based filtering; admin-only en creación/edición
Facturas	GET/PUT /facturas/{id}	Consulta o cambio de facturas de otros proveedores/áreas	Role-based access; validación por flujo financiero
Documentos adjuntos	GET /documentos?factura_id=X, POST adjuntos	Acceso a documentos de factura ajena	Ownership check sobre factura
Comentarios factura	GET/POST comentarios	Acceso a conversaciones de facturas fuera de ámbito	Validación de acceso a factura previa a operar
Historial factura	GET historial por factura	Exposición de trazabilidad financiera	Acceso condicionado por rol y factura
Parámetros SLA	POST/PUT /parametros-sla	Modificación de configuración por usuarios no admin	Admin-only
Reportes	GET /reportes	Exposición de datos financieros agregados	User-scoped / rol autorizado
Centros de costo / cuentas contables	CRUD catálogos financieros	Manipulación de catálogos	Admin financiero/admin sistema

6.5 Préstamos públicos y token anti-IDOR

El flujo público no debe confundirse con endpoints internos. Lo público debe limitarse a crear solicitud con reCAPTCHA y consultar/actualizar solo con token público válido.

Prueba	Resultado esperado	Riesgo si falla
POST /prestamos/public/solicitar/ sin recaptcha_token	403 si RECAPTCHA_SECRET_KEY está configurado	Creación automatizada de solicitudes públicas
POST /prestamos/public/solicitar/ con token válido	201 + token_publico	No aplica
GET /prestamos/public/<id>/ sin X-Public-Token	403	Enumeración de solicitudes públicas
GET /prestamos/public/<id>/ con token incorrecto	403	IDOR por token cruzado
GET solicitud A con token de solicitud B	403	Acceso a solicitud ajena
PUT/DELETE sin token	403	Modificación/eliminación no autorizada

7. Protección CSRF

La auditoría CSRF identificó uso extensivo de @csrf_exempt y mutaciones sin protección central. Se implementó protección compatible con JSON API y SessionAuthentication.

Elemento	Detalle
Hallazgo inicial	170 instancias de @csrf_exempt en backend
Riesgo	POST/PUT/PATCH/DELETE ejecutables desde sitios maliciosos si la sesión del usuario estaba activa
Mitigación principal	JSONCsrfMiddleware con validación de X-CSRFToken
Endpoint de token	GET /api/csrf-token/
Métodos protegidos	POST, PUT, PATCH, DELETE
Métodos permitidos sin token	GET, HEAD, OPTIONS, TRACE
Respuesta esperada sin token	403 Forbidden con mensaje de token faltante
Compatibilidad	JSON API + SessionAuthentication + cookie csrftoken

7.1 Archivos con mayor exposición inicial

Archivo	Instancias @csrf_exempt detectadas
backend/espacios/views.py	29
backend/horario/views.py	26
backend/usuarios/views.py	14
backend/prestamos/views.py	12
backend/espacios/api_views.py	11
backend/asignaturas/views.py	10
backend/componentes/views.py	10
backend/recursos/views.py	10
backend/horario/api_views.py	9
backend/notificaciones/views.py	9
backend/periodos/views.py	7
backend/facultades/views.py	5
backend/grupos/views.py	5
backend/programas/views.py	5
backend/sedes/views.py	5
backend/chatbot/views.py	3

7.2 Flujo de validación CSRF

1. Cliente solicita GET /api/csrf-token/
2. Backend devuelve { "csrfToken": "..." }
3. Frontend almacena el token de forma controlada
4. Cada mutación envía header X-CSRFToken
5. JSONCsrfMiddleware valida:
 - Si método seguro: permite
 - Si endpoint explícitamente público: permite
 - Si método de escritura sin token: 403
 - Si token inválido: 403
 - Si token válido: continúa a la vista

7.3 Pruebas CSRF recomendadas para cierre

Prueba	Comando / acción	Resultado esperado
POST sin token	curl -X POST http://localhost:8000/usuarios/ -H 'Content-Type: application/json' -d '{}'	403
GET sin token	GET /usuarios/list/	200 si la lógica de negocio lo permite
POST con token	GET /api/csrf-token/ y luego POST con X-CSRFToken	201/200 o error funcional, no error CSRF
Frontend	Crear/editar/eliminar recurso desde UI	Debe enviar X-CSRFToken en Network

8. Protección XSS

La protección XSS se centró en sanitizar entradas de usuarios tanto en vistas legacy como en serializers DRF, especialmente en módulos académico, financiero, usuarios, préstamos, componentes y notificaciones.

Módulo	Endpoints/serializers	Campos protegidos	Estado
Académico	Grupos, Asignaturas, Programas, Facultades, Sedes, Recursos, Horarios	30+	100%
Financiero	Proveedor, Departamento, Cuenta Contable, Centro de Costo, Factura	40+	100%
Usuarios	create_rol, update_rol,	nombre, descripción, correo, sede	100%

	create_usuario, update_usuario		
Préstamos	create_tipo_actividad, create_prestamo, create_prestamo_publico	motivo, teléfono, identificación, correo, nombre completo	100%
Componentes/Notificaciones	Serializers y legacy views revisados post-merge	mensaje, tipo_notificacion, nombre, descripción	Mitigado en revisión post-merge

8.1 Arquitectura de sanitización

Componente	Implementación
Helper central	backend/mysite/xss_protection.py
Funciones	sanitize_string, sanitize_integer, sanitize_boolean, sanitize_dict, sanitize_filename, escape_html_output
Esquemas	GRUPO_SCHEMA, ASIGNATURA_SCHEMA, PROGRAMA_SCHEMA, FACULTAD_SCHEMA, SEDE_SCHEMA, RECURSO_SCHEMA, HORARIO_SCHEMA, PROVEEDOR_SCHEMA, FACTURA_SCHEMA, DEPARTAMENTO_SCHEMA, CUENTA_CONTABLE_SCHEMA, CENTRO_COSTO_SCHEMA, ROL_SCHEMA, USUARIO_SCHEMA, TIPO_ACTIVIDAD_SCHEMA, PRESTAMO_SCHEMA, PRESTAMO_PUBLICO_SCHEMA
Estrategia	Whitelists por tipo de campo + longitud máxima + validación de tipos + rechazo de patrones peligrosos

8.2 Vectores XSS probados/bloqueados

Vector	Ejemplo	Control aplicado
Script injection	<script>alert('SIHUL')</script>	Sanitización y eliminación/bloqueo de tags
Event handlers		Bloqueo de onerror/onload/onclick
JavaScript protocol	javascript:alert(1)	Whitelist y rechazo de protocolo
SVG/XML	<svg onload=alert(1)>	Bloqueo de tags peligrosos
Data URI	data:text/html,<script>...	Rechazo por patrón
HTML entity bypass	<script>	Normalización/sanitización
Unicode/URL encoding	%3Cscript%3E	Detección de patrones codificados
Case variation	<ScRiPt>alert(1)</sCrIpT>	Búsqueda case-insensitive
Attribute breaking	" onclick="alert(1)	Whitelist por campo

8.3 Campos sanitizados destacados

Módulo	Campos
Académico	nombre, código, tipo, horas, dirección, descripción, día_semana, hora_inicio, hora_fin
Financiero	nit, razón_social, nombre_comercial, dirección, ciudad, departamento, email, teléfono, contacto_principal, observaciones, número_factura, tipo_documento, cuenta_bancaria_proveedor
Usuarios	nombre, correo, rol, sede, descripción
Préstamos	nombre_completo, correo, teléfono, identificación, motivo, tipo_actividad
Notificaciones	mensaje, tipo_notificacion, prioridad
Componentes	nombre, descripción

9. Carga de archivos y manejo seguro de adjuntos

La revisión de carga de archivos se concentró en el módulo financiero, especialmente DocumentoAdjunto, para evitar archivos maliciosos, path traversal, filtración de nombres y ejecución de contenido no permitido.

Control	Implementación
Extensiones permitidas	pdf, xml, png, jpg, jpeg
Tamaño máximo	10 MB
MIME type	Verificación del tipo declarado y real
Magic bytes	Validación de encabezados PDF/PNG/JPG/XML
HTML camuflado	Rechazo de contenido HTML renombrado como PDF/XML
Nombre seguro	Sanitización de nombre original y almacenamiento con UUID
Path traversal	Evitar uso del nombre original como ruta ejecutable
Integridad	Debe existir archivo o url_storage; url_storage debe ser http/https o vacío

9.1 Casos de prueba de carga de archivos

Archivo probado	Resultado esperado	Estado
documento.pdf válido	Aceptado si cumple MIME/magic bytes/tamaño	Aprobado/validar
imagen.jpg/png válida	Aceptado si cumple	Aprobado/validar
script.js	Rechazado	Mitigado
archivo.html	Rechazado	Mitigado
shell.php	Rechazado	Mitigado
imagen.jpg.php	Rechazado por doble extensión	Mitigado
archivo_muy_pesado.pdf >10MB	Rechazado	Mitigado
HTML renombrado a .pdf	Rechazado por MIME/magic bytes	Mitigado

10. SQL Injection y validación de consultas

La aplicación usa ORM en la mayoría de flujos. El hallazgo puntual crítico fue un comando de sincronización Oracle que aceptaba consulta cruda desde CLI.

Elemento	Detalle
Hallazgo	Comando de sincronización Oracle ejecutaba query cruda con cursor.execute(query)
Impacto	Posible ejecución de sentencias no autorizadas si alguien con acceso al comando suministraba SQL malicioso
Mitigación	validate_query(): solo SELECT, bloqueo de DROP/DELETE/UPDATE/INSERT/EXEC/EXECUTE, bloqueo de múltiples sentencias y comentarios SQL
Archivo	backend/usuarios/management/commands/sincronizar_oracle.py
Estado	Mitigado

10.1 Payloads de prueba generales

Payload	Uso	Resultado esperado
'	Formularios/filtros/búsquedas	No stack trace, no SQL error visible
"	Formularios/filtros/búsquedas	Respuesta controlada
1 OR 1=1	Filtros numéricos o texto	No ampliar resultados indebidamente
'; DROP TABLE usuarios; --	Validación defensiva	Rechazo/control, nunca ejecución

11. Dependencias, CI/CD y vulnerabilidades de paquetes

Se detectaron fallos de auditoría de dependencias en GitHub Actions. El comportamiento de los workflows fue correcto: fallaban al encontrar vulnerabilidades HIGH/CRITICAL.

Componente	Vulnerabilidades detectadas	Paquetes	Mitigación
Backend	11	unicorn, werkzeug, cryptography	Actualización de versiones seguras y pins en requirements
Chatbot	9	python-dotenv, python-multipart, langchain-text-splitters, starlette	Upgrade FastAPI/Starlette/Uvicorn y restauración de dependencias faltantes
Frontend	1	xlsx	Seguimiento por ausencia de fix disponible; evaluar reemplazo o mitigación de uso
CI	2 checks fallando	pip-audit, safety, npm audit	Mantener falla intencional para

			HIGH/CRITICAL; validar verde tras mitigación
--	--	--	--

11.1 Dependencias clave corregidas

Paquete	Versión vulnerable/previa	Versión objetivo/documentada	Motivo
gunicorn	21.2.0	22.0.0	Security patches
werkzeug	3.0.3	3.1.6	Corrección CVEs acumulados
cryptography	42.0.8	46.0.6	Correcciones de seguridad/FIPS
fastapi	0.115.x / intermedias	0.136.0	Compatibilidad con Starlette segura
starlette	0.41.x / 0.48.x / 0.52.x	0.55.0 o 1.0.1 según consolidación	Eliminar CVEs Starlette
uvicorn	faltante	0.34.0	Permitir inicio del chatbot
xlsx	vulnerable HIGH	Sin fix npm según reporte	Riesgo residual: monitorear o reemplazar

11.2 Validaciones recomendadas

```
# Backend
docker compose exec backend pip-audit
docker compose exec backend safety check -r backend/requirements.txt
# Chatbot
docker compose exec chatbot pip-audit -r requirements.txt
# Frontend
cd frontend
npm audit --omit=dev --audit-level=high
npm outdated
```

12. Secretos, configuración y endurecimiento Docker/Nginx

Se detectaron secretos y credenciales hardcodeadas. La mitigación consistió en trasladarlas a variables de entorno y reforzar .gitignore y settings por entorno.

Hallazgo	Mitigación
POSTGRES_PASSWORD, RECAPTCHA_SITE_KEY, RECAPTCHA_SECRET_KEY y PGADMIN_DEFAULT_PASSWORD en docker-compose	Uso de variables de entorno y .env no versionado
DATABASE_URL inline en chatbot	Uso de chatbot/.env o variable externa
DJANGO_SECRET_KEY no estricta en producción	settings.py exige DJANGO_SECRET_KEY cuando ENVIRONMENT=production
.env con riesgo de commit	.gitignore reforzado para .env, *.env, claves, node_modules, dist, logs
Producción sin flags de seguridad	Activación por ENVIRONMENT=production: SSL/HSTS/secure cookies/headers/CORS/CSRF origins
ZAP Baseline no automatizado inicialmente	docker-compose.zap.yml y workflow zap-baseline.yml

12.1 Variables que deben confirmarse antes de producción

- ENVIRONMENT=production
- ALLOWED_HOSTS
- CORS_ALLOWED_ORIGINS
- CSRF_TRUSTED_ORIGINS
- DJANGO_SECRET_KEY
- DB_HOST/DB_NAME/DB_USER/DB_PASSWORD
- RECAPTCHA_SITE_KEY
- RECAPTCHA_SECRET_KEY
- CHATBOT_FASTAPI_URL
- VITE_API_URL
- VITE_CHATBOT_URL

Riesgo residual crítico

Toda clave real que haya estado previamente en docker-compose, logs o historial de Git debe rotarse. Mover secretos a .env evita nuevas exposiciones, pero no invalida claves ya filtradas.

13. OWASP ZAP, headers y controles perimetrales

Se usó ZAP Desktop durante pruebas y se dejó preparado un flujo ZAP Baseline para ejecución local/CI. El escaneo de frontend directo Vite fue útil para alertas iniciales, pero el objetivo final debe ser Nginx/staging.

Control	Estado / Acción
ZAP Desktop	Ejecutado inicialmente contra frontend; alertas de CSP, anti-clickjacking, X-Content-Type-Options, server disclosure, información en URL y comentarios
ZAP Baseline local	docker-compose.zap.yml preparado; pendiente ejecutar y revisar reporte final
ZAP CI	Workflow zap-baseline.yml preparado contra http://localhost:8081
CSP	Debe validarse y ajustar en Nginx según recursos reales del frontend
X-Frame-Options / frame-ancestors	Debe bloquear clickjacking
X-Content-Type-Options	nosniff requerido
Referrer-Policy	Recomendado strict-origin-when-cross-origin o similar
Permissions-Policy	Reducir APIs del navegador no usadas
HSTS	Activar solo cuando HTTPS esté estable

13.1 Comandos ZAP sugeridos

```
docker compose up -d db backend frontend nginx
docker compose -f docker-compose.yml -f docker-compose.zap.yml run --rm zap
# Revisar:
zap-reports/zap-baseline.html
```

14. Matriz detallada de hallazgos

ID	Categoría	Sev.	Hallazgo	Cómo se mitigó	Estado
H-001	Auth/Config	Crítica	Login no funcional por VITE_API_URL con comillas que generaban URL inválida	Remove comillas en docker-compose y rebuild frontend	Resuelto
H-002	Auth/CSRF	Crítica	Login/reCAPTCHA/logout bloqueados por middleware CSRF	Allowlist de endpoints públicos y respeto a decoradores csrf_exempt	Resuelto
H-003	Auth	Alta	/api/usuarios/login/ permitía intentos ilimitados	Lockout en endpoint DRF; bloqueo 2 minutos; respuesta 429/bloqueo	Resuelto
H-004	UX/Auth	Media	Bloqueo podía evadirse al recargar UI	localStorage lockout_until, contador y botón deshabilitado	Resuelto
H-005	Sesión	Info	Cookie Secure no validable en HTTP local	Documentar pendiente y validar en HTTPS/staging	Pendiente HTTPS
H-006	API/Usuarios	Crítica	Docente veía todos los usuarios	Queryset restringido: no admin solo propio registro	Resuelto
H-007	API/Usuarios	Crítica	Exposición de contraseña_hash	Eliminar del serializer/respuesta; perfil mínimo	Resuelto
H-008	Autorización	Crítica	Docente podía crear usuarios	Permisos admin-only para create/update/delete	Resuelto
H-009	API expuesta	Alta	Catálogos académicos visibles sin login	Eliminar AllowAny; isAuthenticated; 403 en incógnito	Resuelto
H-010	Autorización	Alta	Docente podía modificar catálogos	Solo lectura para usuario normal; escritura admin-only	Resuelto
H-011	Horarios	Alta	Horarios/reportes públicos con información sensible	Exigir autenticación y controlar usuario_id	Resuelto
H-012	Préstamos públicos	Alta	GET /api/prestamos/publicos/ público devolvía solicitudes	GET requiere auth; POST público solo con reCAPTCHA	Resuelto
H-013	Préstamos públicos	Alta	Mis Solicitudes devolvía registros ajenos	Filtrar por identificación+correo; solo admin lista todo	Resuelto
H-014	IDOR	Crítica	Karol veía préstamos de otro docente por ID 3/4	Filtrar PrestamoEspacio por usuario; admin ve todos	Resuelto
H-015	DRF UI	Media	Browsable API mostraba formularios	DEFAULT_RENDERER_CLASSES = JSONRenderer	Resuelto
H-016	IDOR Académico	Alta	Endpoints /espacios/* legacy exponían datos internos	Auth obligatoria; escritura admin-only; ownership/supervisor	Resuelto
H-017	IDOR Financiero	Crítica	Facturas/documentos/proveedores accesibles por IDs/roles no validados	Role-based filtering y ownership validation	Resuelto
H-018	CSRF	Crítica	170 @csrf_exempt con mutaciones sin protección	JSONCsrfMiddleware + X-CSRFToken + /api/csrf-token/	Resuelto
H-019	XSS	Crítica	Inputs sin sanitización robusta	xss_protection.py, sanitize_dict, schemas por módulo	Resuelto
H-020	Uploads	Crítica	Adjuntos financieros podían aceptar archivos peligrosos	Ext/MIME/magic bytes/tamaño/UUID/sanitización	Resuelto
H-021	SQLi	Crítica	sincronizar_oracle.py ejecutaba query cruda	Validar solo SELECT y bloquear keywords peligrosas	Resuelto
H-022	Secrets	Crítica	Secretos hardcoded en docker-compose	Variables de entorno, .env, .gitignore, .env.example	Mitigado; rotar claves
H-023	Dependencias	Crítica	21 vulnerabilidades HIGH/CRITICAL en pip/npm	Upgrade/pins; pip-audit/safety/npm audit en	Mitigado

				CI	
H-024	Chatbot	Crítica	Uvicorn/FastAPI faltantes y Starlette vulnerable	Agregar dependencias y actualizar versiones	Resuelto
H-025	pgAdmin	Alta	Contenedor en restart loop por correo inválido .local	Cambiar a admin@localhost.com	Resuelto
H-026	ZAP/Headers	Alta	Headers de seguridad pendientes de validación final	ZAP Baseline local/CI; ajustar Nginx según reporte	Pendiente ejecución final
H-027	Logs/Auditoría	Media	Necesidad de trazabilidad de eventos de seguridad	Recomendación de logs para 401/403/CSRF/XSS/uploads	Recomendado

15. Matriz de pruebas realizadas y pruebas de cierre

ID	Prueba	Usuario/Contexto	Resultado esperado	Estado
T-001	Login válido	admin/docente/proveedor	200 y sesión/token válido	Aprobada
T-002	Login inválido	correo válido/clave errada	401 mensaje genérico	Aprobada
T-003	Intentos fallidos repetidos	mismo usuario/IP	Bloqueo 2 min	Aprobada
T-004	Recarga durante bloqueo	UI login	Bloqueo persiste	Aprobada
T-005	Cookie sessionid	después de login	HttpOnly + SameSite=Lax	Aprobada local
T-006	Logout	usuario autenticado	/api/auth/user/ no autenticado	Aprobada
T-007	Django admin con usuario normal	docente	redirect/login/403	Aprobada
T-008	GET /api/usuarios/ propio	Karol 652	200 perfil mínimo	Aprobada
T-009	GET /api/usuarios/ admin ajeno	Karol -> id 2	403/404	Aprobada
T-010	POST usuario como docente	docente	403/no formulario	Aprobada
T-011	Catálogos en incógnito	sin login	403	Aprobada
T-012	Catálogos como docente	docente	lectura filtrada; sin escritura	Aprobada
T-013	Préstamo propio	Karol -> préstamos 1/2	200	Aprobada
T-014	Préstamo ajeno	Karol -> préstamos 3/4	403/404	Aprobada tras corrección
T-015	GET /api/prestamos/publicos/ incógnito	sin login	403	Aprobada tras corrección
T-016	Mis Solicitudes	usuario público/normal	solo identificación+correo coincidente	Aprobada tras corrección
T-017	CSRF POST sin token	curl/API	403	Pendiente validar final
T-018	CSRF POST con token	curl/API	2xx o error funcional	Pendiente validar final
T-019	XSS payload script	formularios críticos	no ejecución	Mitigado/validar regresión
T-020	XSS event handler	observaciones/motivos	no ejecución	Mitigado/validar regresión
T-021	SQLi payload simple	filtros/búsquedas	sin stack trace ni error SQL	Mitigado/validar regresión
T-022	Upload .php/.html	.php/.html/.js	rechazo	Mitigado/validar regresión
T-023	Upload >10MB	adjunto financiero	rechazo	Mitigado/validar regresión
T-024	ZAP Baseline	http://localhost:8081	sin High/Critical	Pendiente ejecución/visión
T-025	pip-audit/npm audit	CI/local	sin High/Critical no exceptuadas	Mitigado/validar CI

16. Riesgos residuales y recomendaciones finales

Prioridad	Recomendación	Motivo
Crítica	Rotar claves reales de DB, reCAPTCHA, pgAdmin y cualquier secreto que haya estado en docker-compose o logs	Mover a .env evita nuevas exposiciones, pero no invalida secretos filtrados
Crítica	Validar en staging/HTTPS con ENVIRONMENT=production	Secure cookies, HSTS, CORS/CSRF origins y ALLOWED_HOSTS no se validan 100% en HTTP local
Alta	Ejecutar ZAP Baseline y corregir todo High/Critical	Confirma headers, cookies, CSP, clickjacking y exposición accidental
Alta	Confirmar workflows CI verdes: Dependency Audit y ZAP Baseline	Evita regresiones en PR/push
Alta	Revisar/reemplazar dependencia xlsx si sigue sin fix	Queda como riesgo residual si el paquete vulnerable sigue en uso
Alta	Implementar secret scanning con gitleaks/trufflehog	Previene nuevos commits con claves
Media	Crear pruebas E2E automatizadas de auth/IDOR/CSRF/XSS/uploads	Evita que cambios futuros rompan mitigaciones
Media	Mejorar logs y alertas de seguridad	Detectar repetidos 401/403, CSRF denegado, uploads rechazados, lockouts
Media	Documentar matriz RBAC oficial	Evita ambigüedades sobre qué rol puede ver/mutar cada recurso
Media	Actualizar guía de despliegue seguro	Asegurar configuración de producción consistente

16.1 Checklist para declarar cierre final

- Ejecutar ZAP Baseline en http://localhost:8081 y anexar reporte.
- Validar CSRF desde frontend con POST sin token y con token.
- Confirmar variables de producción y ALLOWED_HOSTS/CORS/CSRF_TRUSTED_ORIGINS.
- Rotar secretos expuestos previamente y documentar fecha de rotación.
- Ejecutar pip-audit, safety y npm audit en rama final.
- Validar que DRF Browsable API esté desactivada en ambiente seguro.
- Ejecutar pruebas IDOR con usuario normal y admin en Académico y Financiero.
- Probar carga de archivos maliciosos y tamaños límite.
- Confirmar logs de eventos de seguridad.
- Generar acta de cierre y aceptación de riesgos residuales.

17. Anexos técnicos

17.1 Comandos útiles

```
# Levantar ambiente local
docker compose up -d
# Ver contenedores
docker ps
# Rebuild frontend para Nginx
cd frontend
npm run build
docker compose restart nginx
# Reiniciar backend
docker compose restart backend
# Consultar IDs de usuario
docker compose exec backend python manage.py shell -c "from usuarios.models import Usuario;
u=Usuario.objects.get(correo='karol.docente@unilibre.edu.co'); print(u.id, u.nombre, u.correo)"
# Consultar préstamos por usuario
docker compose exec backend python manage.py shell -c "from prestamos.models import PrestamoEspacio;
print(list(PrestamoEspacio.objects.values_list('id','usuario_id')[:10]))"
```

17.2 URLs críticas de prueba

Categoría	URL	Esperado
Usuario propio	http://localhost:8000/api/usuarios/652/	200
Usuario ajeno/admin	http://localhost:8000/api/usuarios/2/	403/404
Préstamo propio	http://localhost:8000/api/prestamos/espacios/1/	200
Préstamo ajeno	http://localhost:8000/api/prestamos/espacios/3/	403/404
Horario propio	http://localhost:8000/horario/mi-horario-docente? usuario_id=652	200
Horario ajeno	http://localhost:8000/horario/mi-horario-docente? usuario_id=5	403/404
Espacios sin login	http://localhost:8000/espacios/list/	403
Préstamos públicos GET	http://localhost:8000/api/prestamos/publicos/	403 sin login
CSRF token	http://localhost:8000/api/csrf-token/	200 + csrfToken
ZAP target	http://localhost:8081	Escaneo baseline

17.3 Fuentes documentales utilizadas

Documento	Uso principal
INFORME_COMPLETO_MITIGACION_VULNERABILIDADES_2026-05-31(1).md	Consolidación de hallazgos, mitigaciones, pruebas y recomendaciones
SIHUL Security Audit and Hardening(1).md	Consolidación de hallazgos, mitigaciones, pruebas y recomendaciones
IDOR_FINAL_AUDIT_REPORT.md	Consolidación de hallazgos, mitigaciones, pruebas y recomendaciones
CSRF_PROTECTION_AUDIT.md	Consolidación de hallazgos, mitigaciones, pruebas y recomendaciones
XSS_BLINDAJE_FINAL_COMPLETO.md	Consolidación de hallazgos, mitigaciones, pruebas y recomendaciones
XSS_BLINDAJE_TOTAL_SIHUL.md	Consolidación de hallazgos, mitigaciones, pruebas y recomendaciones
ANALISIS_FAILING_CHECKS.md	Consolidación de hallazgos, mitigaciones, pruebas y recomendaciones
DEPENDENCY_VULNERABILITY_EXCEPTIONS.md	Consolidación de hallazgos, mitigaciones, pruebas y recomendaciones
DIAGNOSTICO_LOGIN_RESUELTO.md	Consolidación de hallazgos, mitigaciones, pruebas y recomendaciones
AUDITORIA_VULNERABILIDADES_POST_MERGE.md	Consolidación de hallazgos, mitigaciones, pruebas y recomendaciones
SECURITY_AUDIT_REPORT_2026-05-30.md	Consolidación de hallazgos, mitigaciones, pruebas y recomendaciones

18. Detalle ampliado endpoint-level y trazabilidad de mitigaciones

Esta sección amplía el informe para dejar trazabilidad operacional de los endpoints, rutas y componentes que fueron probados o endurecidos. No todos los endpoints requieren el mismo nivel de detalle en producción, pero se documentan para demostrar la cobertura de las pruebas y facilitar futuras auditorías.

18.1 Endpoints de autenticación, sesión y control de acceso

Endpoint / componente	Método	Problema o riesgo	Implementación / mitigación	Prueba de validación
/api/usuarios/login/	POST	Endpoint DRF real sin logout y bloqueado por CSRF en etapa inicial	CSRF allowlist controlado + logout backend 2 minutos + cache + respuesta controlada	Credencial inválida repetida -> bloqueo; credencial válida -> 200
/usuarios/login/	POST	Endpoint legacy con lógica distinta al endpoint real del frontend	Alinear comportamiento de autenticación y protección; mantener controlado	Validar que no quede flujo sin logout
/api/usuarios/logout/	POST	Debe cerrar sesión sin quedar bloqueado por CSRF indebidamente	CSRF allowlist para logout y session flush	Luego de logout, endpoint de usuario debe responder no autenticado
/api/auth/logout/ o /auth/logout/	POST	Cierre de sesión en endpoint alternativo	Ruta incluida en lista de rutas públicas controladas cuando aplica	Logout efectivo
/api/csrf-token/	GET	Necesario para mutaciones protegidas por CSRF	Endpoint controlado para emitir token de sesión	GET devuelve csrfToken
/usuarios/session-auth-state/	GET	Confusión con /api/auth/user/ por mecanismo de sesión usado	Usar este endpoint para validar sesión SIHUL cuando aplica	200 cuando sesión SIHUL existe
/admin/	GET	Panel administrativo sensible	Acceso solo a usuarios admin; recomendar restricción por red/VPN/MFA	Usuario normal -> login/403
Frontend Login.tsx / useLogin.ts	UI	Bloqueo visual evadible por recarga	Persistencia localStorage logout_until + contador + botón disabled	Recargar no permite reintentos hasta expirar

18.2 Endpoints académicos corregidos o endurecidos contra IDOR

Módulo	Endpoint/acción	Riesgo IDOR	Control aplicado	Resultado esperado
Usuarios	/api/usuarios/	Listaba usuarios de todo el sistema	Queryset por usuario; admin ve todos	No admin solo ve su perfil mínimo
Usuarios	/api/usuarios/{id}/	Ver usuario ajeno por ID	same-user-or-admin	Propio -> 200; ajeno -> 403/404
Usuarios	/usuarios/list/	Legacy list expuesto	auth + admin/full or propio	No admin no lista todos
Usuarios	/usuarios/get/{id}/	Legacy get permitía ID ajeno	same-user-or-admin	Ajeno -> 403/404
Usuarios	/usuarios/create/update/delete	Mutación por rol no autorizado	admin-only	Docente -> 403
Grupos	/grupos/list/	Listado sin filtro	auth + seccional filtering	Solo seccional autorizada
Grupos	/grupos/{id}/	Consulta por ID de otra seccional	auth + filtro	No autorizado -> 403/404
Grupos	POST /grupos	Creación por usuario no admin	admin-only	Docente -> 403
Grupos	PUT /grupos/{id}/	Edición por usuario no admin	admin-only	Docente -> 403
Grupos	DELETE /grupos/{id}/	Eliminación por usuario no admin	admin-only	Docente -> 403
Asignaturas	/asignaturas/list/	Listado no filtrado	auth + seccional filtering	Solo autorizadas
Asignaturas	/asignaturas/{id}/	Consulta de asignatura ajena	auth + filtro	403/404 si no corresponde
Asignaturas	POST /asignaturas	Creación no autorizada	admin-only	403
Asignaturas	PUT /asignaturas/{id}/	Edición no autorizada	admin-only	403
Asignaturas	DELETE /asignaturas/{id}/	Eliminación no autorizada	admin-only	403
Programas	/programas/list/	Programas de otras seccionales	seccional filtering	Solo ámbito del usuario
Programas	/programas/{id}/	Consulta por ID no autorizado	auth + filtro	403/404
Programas	POST/PUT/DELETE /programas	Mutación no autorizada	admin-only	403 para no admin
Facultades	/facultades/list/	Facultades de otras sedes/seccionales	seccional filtering	solo autorizadas
Facultades	/facultades/{id}/	Consulta por ID ajeno	auth + filtro	403/404
Facultades	POST/PUT/DELETE /facultades	Mutación no autorizada	admin-only	403
Sedes	/sedes/list/	Sedes no filtradas	auth + seccional filtering	solo autorizadas
Sedes	/sedes/{id}/	Consulta sede ajena	auth + filtro	403/404 si no corresponde
Sedes	POST/PUT/DELETE /sedes	Mutación no autorizada	admin-only	403
Espacios	/api/espacios/	Listado de espacios internos	auth + seccional filtering	datos permitidos
Espacios	/api/espacios/{id}/	Consulta de espacio por ID	auth + filtro	403/404 si no corresponde
Espacios	/espacios/list/	Legacy público	auth obligatoria	incógnito -> 403
Espacios	/espacios/{id}/	Legacy exponía ubicación/capacidad/recursos	auth + control	sin login -> 403
Espacios	/espacios/permitido/*	Acceso a permisos de espacios por usuario	admin o dueño	ajeno -> 403
Espacios	/espacios/horarios/*	Horarios de espacios/supervisor	auth + supervisor/dueño/admin	no autorizado -> 403
Horarios	/horario/mi-horario-docente? usuario_id=X	Cambio de usuario_id	same-user-or-admin	ajeno -> 403/404
Horarios	/horario/mi-horario-estudiante? usuario_id=X	Cambio de usuario_id	same-user-or-admin	ajeno -> 403/404
Horarios	/api/horario/por-periodo/	Consulta de horarios sin auth	auth obligatoria	sin login -> 403
Horarios	/api/horarios/exportar-pdf-docente/	Exportación de otro docente	same-user-or-admin	ajeno -> 403
Horarios	/api/horarios/exportar-excel-docente/	Exportación de otro docente	same-user-or-admin	ajeno -> 403
Préstamos Académicos	/api/prestamos/espacios/	Listado no filtrado por usuario	queryset por usuario; admin ve todos	usuario normal solo propios
Préstamos Académicos	/api/prestamos/espacios/{id}/	Karl veía préstamos 3/4 de otro docente	ownership check	ajeno -> 403/404
Préstamos Académicos	/api/prestamos/recursos/	Recursos por préstamo ajeno	filtrado por prestamo usuario_id	ajeno -> vacío/403
Préstamos Académicos	/prestamos/list/	Legacy potencialmente público	auth obligatoria	incógnito -> 403
Préstamos Académicos	/prestamos/public/mis-solicitudes/	Mis solicitudes sin filtro correcto	identificación+correo / token	solo coincidentes

18.3 Endpoints financieros corregidos o endurecidos contra IDOR/RBAC

Módulo	Endpoint/acción	Riesgo	Control aplicado	Resultado esperado
Proveedores	GET /api/financiero/proveedores/	Proveedor/funcionario veía datos fuera de ámbito	role-based filtering	Proveedor ve solo lo suyo; admin ve todos
Proveedores	GET /api/financiero/proveedores/{id}/	Consulta de proveedor ajeno por ID	ownership/role check	no autorizado -> 403/404
Proveedores	POST/PUT/DELETE proveedores	Mutación por rol no permitido	admin-only / rol autorizado	no autorizado -> 403
Facturas	GET /api/financiero/facturas/	Listado no filtrado	role-based queryset	según rol
Facturas	GET /api/financiero/facturas/{id}/	Ver factura ajena por ID	role-based access	403/404
Facturas	PUT/PATCH factura	Cambio de factura por rol no autorizado	role-based update	403 si no corresponde
Documentos Adjuntos	GET /api/financiero/documentos/? factura_id=X	Descarga/listado de documentos de factura ajena	validar acceso a factura	403/404
Documentos Adjuntos	POST documento	Carga a factura sin permiso	ownership check de factura	403
Comentarios Factura	GET/POST comentarios	Ver o comentar factura ajena	validar acceso a factura	403
Historial Factura	GET historial	Exposición de trazabilidad financiera	role-based access	403 si no corresponde
Reportes	GET reportes	Exposición agregada	user-scoped / rol autorizado	según rol

Parámetros SLA	POST/PUT /parametros-sla	Manipulación de SLA	admin-only	403 para no admin
Departamentos	CRUD departamentos	Manipulación catálogo financiero	admin financiero/sistema	403 para no autorizado
Cuentas Contables	CRUD cuentas	Manipulación contable	admin financiero/sistema	403
Centros de Costo	CRUD centros	Manipulación centro de costo	admin financiero/sistema	403

18.4 Endpoints públicos permitidos y endpoints públicos cerrados

Endpoint	Estado final	Motivo / observación
GET /api/prestamos/tipos-actividad/	Público permitido	Solo debe exponer catálogo mínimo para flujo público
GET /api/prestamos/public/espacios-disponibles/	Público permitido	Permite consultar disponibilidad mínima; no debe exponer datos internos innecesarios
GET /prestamos/public/espacios-disponibles/	Público permitido si corresponde al flujo legacy	Misma restricción de datos mínimos
POST /api/prestamos/publicos/	Público con reCAPTCHA	Creación de solicitud pública; debe validar recaptcha_token
POST /prestamos/public/solicitar/	Público con reCAPTCHA	Debe devolver token_publico si es válido
GET /api/prestamos/publicos/	Cerrado	No debe listar solicitudes sin auth
GET /api/prestamos/publicos/{id}/	Cerrado o admin-only	Detalle no debe ser público
GET /prestamos/public/{id}/	Token requerido	Debe exigir X-Public-Token correcto
PUT /prestamos/public/update/	Token requerido	Solo dueño del token puede actualizar
DELETE /prestamos/public/delete/	Token requerido	Solo dueño del token puede eliminar/cancelar
GET /horario/por-periodo/	Cerrado	Horario es información interna
GET /espacios/*	Cerrado	Espacios, horarios, recursos, capacidad y ubicación son datos internos

18.5 Resumen endpoint-level de XSS por esquema

Schema	Módulo	Campos/uso principal	Endpoints relacionados
GRUPO_SCHEMA	Académico	nombre, grupo, docente, programa	create/update grupos
ASIGNATURA_SCHEMA	Académico	nombre, código, tipo, horas	create/update asignaturas
PROGRAMA_SCHEMA	Académico	nombre, facultad, estado	create/update programas
FACULTAD_SCHEMA	Académico	nombre, sede	create/update facultades
SEDE_SCHEMA	Académico	nombre, dirección, ciudad	create/update sedes
RECURSO_SCHEMA	Académico	nombre, descripción	create/update recursos
HORARIO_SCHEMA	Académico	día_semana, horas, docente, espacio	create/update horarios
PROVEEDOR_SCHEMA	Financiero	nit, razón_social, dirección, email, observaciones	ProveedorSerializer
FACTURA_SCHEMA	Financiero	numero_factura, tipo_documento, descripción, observaciones	FacturaCreateSerializer
DEPARTAMENTO_SCHEMA	Financiero	nombre, descripción	DepartamentoSerializer
CUENTA_CONTABLE_SCHEMA	Financiero	codigo, nombre, descripción	CuentaContableSerializer
CENTRO_COSTO_SCHEMA	Financiero	codigo, nombre, descripción	CentroCostoSerializer
ROL_SCHEMA	Usuarios	nombre, descripción	create rol/update rol
USUARIO_SCHEMA	Usuarios	nombre, correo, sede	create usuario/update usuario
TIPO_ACTIVIDAD_SCHEMA	Préstamos	nombre, descripción	create_tipo_actividad
PRESTAMO_SCHEMA	Préstamos	motivo, teléfono	create_prestamo
PRESTAMO_PUBLICO_SCHEMA	Préstamos público	nombre_completo, correo, teléfono, identificación, motivo	create_prestamo_publico
COMPONENTE_SCHEMA	Componentes	nombre, descripción	ComponenteSerializer
NOTIFICACION_SCHEMA	Notificaciones	tipo_notificacion, mensaje, prioridad	NotificacionSerializer / legacy

19. Bitácora de implementación por vulnerabilidad

Esta bitácora resume qué se implementó técnicamente, de forma trazable y orientada a revisión de código.

Vulnerabilidad	Qué se implementó	Dónde aplica	Resultado
Auth logout	Contador de intentos, cache, bloqueo temporal, 429/control de UI	Login DRF y legacy	Mitiga fuerza bruta básica
Persistencia logout	localStorage logout_until + contador visible + botón disabled	Frontend Login	Evita bypass por recarga
CSRF	JSONCsrfMiddleware, /api/csrf-token/, X-CSRFToken obligatorio	Mutaciones JSON/SessionAuthentication	Mitiga CSRF en 170+ endpoints
IDOR	get_queryset filtrado, same-user-or-admin, admin-only, role-based filtering	Académico y financiero	Mitiga acceso por cambio de ID
XSS	sanitize_dict + schemas + whitelists + validate() en serializers	Formularios/JSON/legacy/DRF	Mitiga scripts y payloads peligrosos
Uploads	Extensión, tamaño, MIME, magic bytes, UUID, sanitización de nombre	DocumentoAdjunto financiero	Mitiga malware/path traversal
Secrets	.env, .gitignore, .env.example, variables Docker, DJANGO_SECRET_KEY obligatorio prod	Docker/settings	Mitiga exposición futura
Dependencias	pip-audit, safety, npm audit, pins, upgrades FastAPI/Starlette	Backend/chatbot/frontend	Mitiga CVEs conocidas
SQLi	validate_query SELECT-only y bloqueo de keywords	Sincronización Oracle	Mitiga ejecución SQL peligrosa
Browsable API	JSONRenderer como render por defecto	DRF	Evita formularios expuestos en API
ZAP	docker-compose.zap.yml + workflow zap-baseline	Nginx local/CI	Verificación continua pasiva
Headers prod	SSL/HSTS/secure cookies/security headers por ENVIRONMENT=production	settings.py/Nginx	Hardening de despliegue

19.1 Validaciones adicionales que debería documentar si aún no están anexas

Prueba adicional	Por qué documentarla	Evidencia sugerida
Ejecución completa de ZAP Baseline	Confirma headers y configuración de frontend/Nginx	HTML report y resumen de alertas
CSRF desde frontend	Asegura que UI envía X-CSRFToken	Capturas Network con/sin token
Pruebas de carga de archivos	Demuestra que no se permite malware camuflado	Capturas 400/errores para .php/.html/>10MB
pip-audit/npm audit final	Comprueba que dependencias están limpias o justificadas	Logs de CI/local
Prueba de roles financiera	Demuestra RBAC financiero por proveedor/contabilidad/tesorería/admin	Matriz 200/403 por rol
Rotación de secretos	Cierra riesgo residual de claves previamente expuestas	Acta o commit/config de nuevas claves
DEBUG/ALLOWED_HOSTS production	Evita errores técnicos y host header issues	settings/env screenshot o checklist DevOps
Logs de seguridad	Permite monitoreo posterior	Ejemplo de log de 401/403/CSRF/upload rejected

20. Resumen final ejecutivo para presentación

Mensaje ejecutivo

Las pruebas y mitigaciones realizadas cubrieron los principales riesgos de seguridad web de SIHUL: autenticación, sesiones, APIs expuestas, IDOR, CSRF, XSS, uploads, SQLi, dependencias, secretos, configuración y escaneo pasivo. El sistema cuenta ahora con controles técnicos aplicados y patrones reutilizables de autorización y sanitización. Para cierre formal se recomienda ejecutar el paquete de validación final en staging/HTTPS, anexar reporte ZAP y evidenciar CI verde. Este informe ampliado debe usarse como documento base de auditoría, sustentación académica/empresarial y guía para futuras revisiones de seguridad.

11. Riesgos residuales y pruebas complementarias recomendadas

Propósito de esta sección. Las pruebas y mitigaciones documentadas en el presente informe permiten sustentar un avance importante en el hardening de SIHUL. Sin embargo, algunas categorías de seguridad requieren validación complementaria en un ambiente staging o producción controlada, especialmente aquellas relacionadas con criptografía, monitoreo, SSRF, deserialización, diseño seguro y pruebas TLS/HTTPS. Esta sección no clasifica dichos puntos como vulnerabilidades confirmadas, sino como controles adicionales recomendados para cierre formal del riesgo residual.

Criterio profesional de clasificación

Los elementos listados a continuación deben tratarse como pruebas complementarias. No deben reportarse como vulnerabilidades explotables hasta que exista evidencia técnica, reproducción del hallazgo, impacto confirmado y validación del equipo responsable del aplicativo.

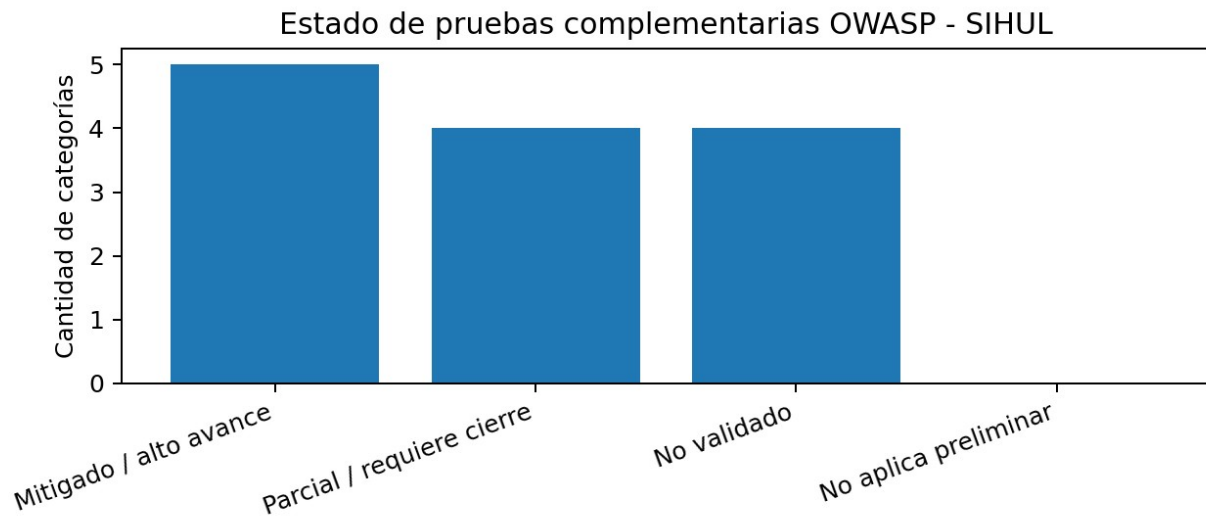


Figura 4. Estado de cobertura de pruebas complementarias OWASP propuestas para SIHUL.

11.1 Matriz de riesgos residuales y pruebas complementarias

ID	Categoría / riesgo	Estado recomendado	Análisis técnico	Decisión para el informe
A1	Broken Access Control	Ampliamente trabajado	IDOR, rutas expuestas, catálogos, préstamos, usuarios, horarios y permisos por rol fueron revisados y mitigados. Mantener verificación regresiva antes de producción.	Incluir como hallazgo principal mitigado y conservar pruebas de regresión.
A2	Cryptographic Failures	No validado completamente	No se puede cerrar al 100% en ambiente local HTTP. Requiere revisar HTTPS real, TLS, HSTS, cookies Secure, certificados, algoritmos de hashing y cifrado de secretos.	Registrar como prueba complementaria en staging/producción.
A3	Injection	Parcialmente mitigado	Se revisó SQLi puntual en sincronización Oracle y validación general de entradas, pero debe ampliarse a búsquedas, filtros, parámetros dinámicos y payloads en APIs.	Incluir como mitigado parcialmente y programar pruebas específicas.
A4	Insecure Design	No evaluado formalmente	Requiere revisión de arquitectura, reglas de negocio, separación de funciones, abuso de flujos y controles de aprobación. Parte fue cubierta con roles/IDOR, pero no toda la categoría.	Registrar como evaluación futura de diseño seguro.
A5	Broken Authentication	Trabajado parcialmente	Se trabajó login, bloqueo por intentos, persistencia del logout, cookies y logout. Falta validar MFA, recuperación de contraseña, política de contraseñas y sesión en HTTPS.	Incluir como hallazgo principal mitigado parcialmente.
A6	Software Integrity	Parcialmente cubierto	Se trabajaron dependencias, auditoría CI y lockfiles. Falta reforzar firma de	Registrar como recomendación de CI/CD y supply chain.

			artefactos, control de integridad de builds, revisión de pipeline y dependencias transitivas.	
A7	Identification and Authentication Failures	Trabajado parcialmente	Se mitigó parte mediante login seguro y controles de sesión. Se recomienda revisar recuperación de cuenta, expiración de sesión, rotación de tokens y usuarios privilegiados.	Incluir dentro de autenticación y gestión de identidad.
A8	Data Validation	Muy avanzado	Se implementaron esquemas de sanitización XSS, validación de uploads, validación de payloads y restricciones por campo en múltiples módulos.	Incluir como control principal mitigado.
A9	Logging and Monitoring	No validado suficientemente	Falta comprobar logs de login fallido, logout, 403, cambios de roles, creación/eliminación, actividad financiera, cargas de archivos y alertas.	Registrar como recomendación crítica antes de producción.
A10	SSRF	No validado	Solo aplica si SIHUL consume URLs externas, webhooks, importaciones remotas, chatbot con enlaces o descargas desde URL. Debe confirmarse con el interesado.	Registrar como prueba futura condicionada al flujo funcional.
B1	Path Traversal	Parcialmente cubierto	Se trabajó en carga de archivos mediante UUID, extensión, MIME y magic bytes. Falta probar traversal en rutas legacy, descarga de archivos, media/static y parámetros filename/path.	Incluir como control parcialmente mitigado y ampliar pruebas.
B2	Weak Crypto	No validado completamente	Requiere revisión de hashes, certificados, TLS, secretos, tokens, claves reCAPTCHA, contraseñas y almacenamiento cifrado. Parte se cubrió en secrets/config.	Registrar como prueba complementaria.
B5	Insecure Deserialization	No evidenciado en fase actual	No se confirmó uso de pickle, YAML inseguro, objetos serializados, XML complejo, payloads binarios o procesamiento inseguro. Debe revisarse código y dependencias.	Registrar como no evidenciado / pendiente confirmar.

11.2 Pruebas complementarias priorizadas

- **Criptografía, TLS y cookies Secure:** Validar HTTPS real en sihul.unilibre.edu.co o staging, certificado vigente, redirección HTTP->HTTPS, HSTS, TLS moderno, ausencia de contenido mixto y cookies sessionId/csrfToken con Secure=True. Esta prueba no se cierra en local HTTP.
- **Logging y monitoreo de seguridad:** Confirmar que queden registrados login fallido, bloqueo por intentos, acceso denegado 403, modificación de usuarios/roles, creación/eliminación de recursos críticos, cargas de archivos, cambios financieros, exportaciones y errores de seguridad. Definir responsables de revisión y retención.
- **SSRF:** Identificar si existen campos que reciban URLs, webhooks, importación remota, lectura de archivos desde URL, integraciones externas o chatbot consumiendo enlaces. Si existen, validar bloqueo de localhost, IPs privadas, metadata endpoints y redirecciones.

- **Deserialización insegura:** Buscar uso de pickle, yaml.load sin SafeLoader, eval, exec, deserialización de objetos, procesamiento XML complejo, archivos comprimidos o payloads binarios. Si se encuentra, aplicar parsers seguros y listas permitidas.
- **Diseño seguro y abuso de reglas de negocio:** Probar abuso de flujos funcionales: aprobaciones sin rol, escalamiento de estados, saltos de etapas, doble aprobación por el mismo usuario, creación masiva, manipulación de montos/fechas/proprietarios y separación de funciones.
- **Path traversal extendido:** Además de uploads, probar endpoints de descarga, rutas media/static, parámetros filename/path, doble extensión, ../, codificación URL, paths absolutos y nombres largos. Verificar que no se puedan leer archivos internos.
- **Supply chain / integridad de software:** Mantener auditoría automática pip-audit, safety y npm audit. Agregar revisión de lockfiles, dependabot/renovate, firmas o checksums de imágenes Docker, control de versiones de base images y revisión de acciones de GitHub.

11.3 Redacción sugerida para presentación al interesado

Aunque la auditoría permitió identificar, corregir y validar vulnerabilidades críticas asociadas a autenticación, control de acceso, IDOR, CSRF, XSS, exposición de APIs, carga de archivos, dependencias y secretos, se recomienda ejecutar una fase complementaria de pruebas avanzadas antes del paso a producción. Esta fase deberá cubrir validaciones criptográficas, logging y monitoreo, SSRF, deserialización insegura, diseño seguro, hardening TLS/HTTPS, pruebas automatizadas y revisión de integridad de software. Estas pruebas complementarias no se clasifican como vulnerabilidades confirmadas hasta contar con evidencia técnica reproducible y validación del equipo responsable.

11.4 Decisión ejecutiva recomendada

Recomendación: presentar el informe actual como evidencia de mitigación de hallazgos confirmados y, en paralelo, dejar aprobado un plan de hardening complementario. Esto evita mezclar hallazgos probados con hipótesis técnicas, protege la calidad del informe y permite priorizar las validaciones que dependen de staging/producción, de infraestructura o de confirmación funcional del interesado.